

“Blood Bank and Donor Locator App”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

SIMRAN JANGID 2520030166

K ANUSRI 2520030432

VENATI RUKESH 2520030519

Under the Esteemed Guidance of

Dr. Spandana. S

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled **“Blood Bank and Donor Locator App”** is a record of Bonafide work of **SIMRAN JANGID - 2520030166, K ANUSRI - 2520030432, VENATI RUKESH - 2520030519** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

SIMRAN JANGID – 2520030166

K ANUSRI – 2520030432

VENATI RUKESH – 2520030519

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **“Blood Bank and Donor Locator App”** is a bonafide work done and submitted by **SIMRAN JANGID - 2520030166, K ANUSRI - 2520030432, VENATI RUKESH - 2520030519** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator **[Course Coordinator Name]**, and our guide **Dr. Spandana. S**, Department of Computer Science and Engineering, whose tremendous support, encouragement, and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P. Venkateshwara Rao**, Head of the *Department of Computer Science and Engineering*, for providing us with adequate facilities, ways, and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and everyone who has assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends, and classmates for their kind help and cooperation during our work.

SIMRAN JANGID – 2520030166

K ANUSRI – 2520030432

VENATI RUKESH – 2520030519

Abstract

Finding a suitable blood donor quickly is one of the most common and most urgent challenges faced by patients and their families during emergencies and planned medical procedures. Traditional methods such as phone calls, social media messages, and personal contacts are slow and do not provide real-time information about donor availability or location. To address this problem, our project focuses on the Design and Development of the Blood Bank and Donor Locator App, an Android-based application that connects blood donors with recipients through a single, centralised platform.

The application allows donors to register and verify their details, including blood group, location, and availability. Recipients can create blood requests and search for compatible nearby donors using geo-location and the Google Maps SDK. Communication between the Android application and the database is handled by REST APIs built with Node.js and Express, and all donor, recipient, blood request, and donation history data is stored in a MySQL database. Firebase Cloud Messaging (FCM) sends instant notifications to matching donors when a request is posted. Security is ensured through password hashing, token-based authentication, and role-based access control.

The system also maintains donation history and provides an admin panel for managing donors, users, and blood requests. The main objective is to reduce the time required to find compatible blood donors and to improve communication between donors and recipients. In summary, this project delivers a reliable and user-friendly digital solution while demonstrating the practical application of database design, backend API development, and distributed system concepts.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives of the Project 1.5 Importance of the Application 1.6 Target Users / Audience	1-5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	6-7
3	Technology Stack	3.1 Front-End Design 3.2 Back-End Design 3.3 Database Design 3.4 Version Control 3.5 APIs or External Services	8-9
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of Each Layer (Frontend, Backend, Database) 4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)	10-11
5	Design	5.2 ER Diagram / Database schema	12-13
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	14-16
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	17-19
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	20-22
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	23-24
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	25-26
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	27-27

S. No.	Chapters	Topics	Page.no
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	28-29
13	References	- Books, tutorials, APIs, documentation sites used	30-31
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Geo Tag photos with guide - Review forms with guide signatures	32-38

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Blood is a life-saving resource that cannot be manufactured; it can only be donated by healthy volunteers. Every day, patients undergoing surgery, accident victims, mothers facing complications during childbirth, and people living with conditions such as thalassemia, sickle cell disease, and cancer depend on a timely supply of blood. Although many people are willing to donate, the gap between a donor who is willing and a patient who is in need is often bridged by chance—a phone call to a friend, a message in a social media group, or a visit to several hospitals in the hope that one of them has the required blood group in stock.

In emergencies, this informal approach costs precious time. A family searching for a rare blood group such as B negative or O negative may have only a few hours to find a donor. Blood banks maintain stock, but stock levels fluctuate and are not visible to the public in real time. Donor registers kept by hospitals, clubs, and volunteer groups are usually maintained on paper or in spreadsheets, are rarely updated, and cannot be searched by location. As a result, a compatible donor living a few kilometres away may never learn that someone nearby needs help.

The widespread availability of smartphones with GPS, mobile internet, and push-notification services has created an opportunity to solve this problem. Android is the most widely used mobile platform in India, and cloud messaging services can deliver an alert to a registered user within seconds. Map services can display donors and hospitals visually, helping a recipient understand who is nearby. Combining these technologies with a reliable backend and a well-designed relational database makes it possible to build a centralised platform in which donors and recipients can find each other quickly.

The idea for this project emerged from this observation. The **Blood Bank and Donor Locator App** is an Android-based application that connects blood donors with recipients who require blood during emergencies or planned medical procedures. Donors register their blood group, location, and availability; recipients post blood requests and search for compatible donors nearby; and the system instantly notifies suitable donors through Firebase Cloud Messaging (FCM). The application maintains donation history and provides an administrator panel to manage users and requests.

From an academic and engineering perspective, the project brings together several core areas covered in the course Database Systems Engineering and Distributed Backend Development:

1. **Database Design and SQL:** modelling donors, recipients, blood requests, blood banks, and donation history as normalised MySQL tables with keys and constraints.
2. **Backend Development:** designing REST APIs with Node.js and Express for registration, login, request handling, and donor searching.

3. **Security:** password hashing, token-based authentication, role-based authorisation, and input validation.
4. **Distributed Systems Concepts:** a client–server architecture in which the Android client, the API server, the database, and the cloud messaging service communicate over the network, with a modular structure that can be split into independent services.
5. **Mobile Application Development:** an Android client built with Java and XML that uses location services and the Google Maps SDK.
6. **Software Engineering Practice:** version control with Git and GitHub, API testing with Postman, and structured documentation.

In summary, the background of this project rests on three observations: finding blood donors today relies on slow and informal channels; modern mobile and cloud technologies make a location-aware, real-time solution practical; and a well-structured backend and database are the key to making such a solution reliable, secure, and scalable.

1.2 Problem Statement

Despite the many people who are willing to donate blood, patients and their families still struggle to locate compatible donors quickly. Information about donors is scattered across hospitals, blood banks, phone contacts, and social media groups. There is no single, up-to-date source that tells a recipient which compatible donors are available nearby right now.

The first difficulty is **speed**. Matching a patient with a donor through phone calls and message forwarding can take hours, while an emergency may allow only a short window. The second difficulty is **accuracy of information**. A donor who donated a month ago, who has moved to another city, or who has changed their phone number may still appear in an old list, and the recipient discovers this only after wasting time on the call.

The third difficulty is **location**. Blood is most useful when the donor can reach the hospital quickly, yet existing methods offer little or no ability to filter donors by distance. The fourth difficulty is **communication**. Even when a donor is willing to help, there is no direct channel through which a recipient can alert all suitable donors at once, so each person must be contacted individually.

Finally, there is a **trust and management** problem. A public donor list can be misused or may contain fake or outdated entries. Without verification and an administrative control point, it is difficult to keep the data reliable and to remove misuse.

Therefore, the problem addressed by this project is the absence of a centralised, location-aware, real-time platform that connects blood donors and recipients, keeps donor availability up to date, notifies suitable donors instantly, and gives administrators control over the data. The proposed application aims to achieve the following resolutions:

1. Provide a single platform where donors register their blood group, location, and availability.
2. Allow recipients to post blood requests and search for compatible donors by blood group and distance.
3. Notify matching donors instantly with a push notification when a request is created.
4. Keep donor information reliable through verification, availability status, and a minimum gap between donations.
5. Maintain donation history and provide an admin panel to manage donors, recipients, and requests.

Existing applications such as Aanjana Rakt Mitra and UBlood show that there is genuine demand for digital blood-donation services. This project focuses on combining location-based donor search, direct notifications to compatible donors, and a clean, well-structured backend in a single Android application.

1.3 Scope of the Project

The scope of the project covers the design and development of an Android application together with its backend services and database. Within this scope, the system provides the following capabilities:

- Registration and login for donors, recipients, and administrators with secure authentication.
- Donor profile management including blood group, age, gender, location, availability, and last donation date.
- Creation, viewing, updating, and cancellation of blood requests by recipients, with urgency levels.
- Searching for compatible donors using blood-group compatibility rules and distance from a chosen location.
- Display of donors, hospitals, and blood banks on a map using the Google Maps SDK.
- Instant push notifications to compatible donors through Firebase Cloud Messaging.
- Donor responses to requests (accept or decline) and tracking of request status.
- Maintenance of donation history for each donor.
- An administrator panel for verifying donors and managing users and blood requests.

The following items are outside the scope of the current version: online payment or sale of blood, real-time inventory synchronisation with hospital information systems, medical screening of donors, and an iOS or web version of the application. These are listed as future enhancements in Chapter 11.

1.4 Objectives of the Project

The primary objective of the project is to reduce the time required to find compatible blood donors and to improve communication between donors and recipients. The specific objectives are:

1. To connect blood donors with recipients through a centralised platform.
2. To reduce the time required to find compatible donors.
3. To enable location-based donor searching using maps and geo-location.
4. To maintain updated donor availability.
5. To provide instant notifications for blood requests.
6. To maintain donation history for donors.
7. To improve communication between donors and recipients.
8. To provide administrative control over donor and request information.
9. To apply database design, REST API development, and security principles in a real-world application.

In essence, the objective of this project goes beyond building a mobile application. It is about demonstrating how a carefully designed database, a secure backend, and modern mobile and cloud services can be combined to address a genuine social need.

1.5 Importance of the Application

The importance of this application lies in the fact that in a medical emergency, time is the most critical resource. A system that can shorten the search for a compatible donor from hours to minutes can directly influence the outcome for a patient. By notifying only those donors who are compatible, available, and nearby, the application avoids flooding people with irrelevant messages and increases the chance of a quick response.

The application also benefits the donor community. Donors receive a transparent record of their donations, are reminded of their eligibility, and can control their own availability. This encourages regular, voluntary donation and builds a reliable pool of donors over time. For hospitals and blood banks, a verified and searchable donor database reduces the burden of making repeated phone calls during an emergency.

From a technical standpoint, the project is important because it demonstrates how the concepts of relational database design, RESTful API development, authentication, and push messaging come together in a complete working system. It also provides a strong base on which features such as AI-based donor matching and hospital integration can be added later.

1.6 Target Users / Audience

The application is designed for the following groups of users:

- **Blood donors:** healthy volunteers who wish to register, keep their availability updated, receive alerts, and track their donation history.
- **Recipients and their families:** patients or relatives who need blood urgently or for planned procedures and want to find compatible donors nearby.
- **Administrators:** authorised staff or team members who verify donors, monitor requests, and remove misuse.
- **Hospitals, blood banks, and volunteer organisations:** institutions that wish to reach a wider pool of donors during shortages or donation drives.

The interface is kept simple and uses clear language, large buttons, and a minimal number of steps so that people of all ages and technical backgrounds can use it, even under stress.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The hardware requirements are divided into the development environment used by the team, the mobile device on which the application runs, and the server on which the backend and database are hosted. The values below are the minimum recommended configurations.

(a) Development machine

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (64-bit)	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB	16 GB (Android Studio and emulator run smoothly)
Storage	20 GB free disk space	256 GB SSD
Display	1366 x 768	1920 x 1080
Network	Broadband internet connection	Stable high-speed connection

(b) Mobile device (end user)

Component	Requirement
Operating System	Android 8.0 (API level 26) or higher
RAM	2 GB or more
Sensors	GPS / location services enabled
Connectivity	Mobile data or Wi-Fi
Services	Google Play Services installed (required for Maps and FCM)

(c) Server

Component	Requirement
Processor	1 vCPU or higher
RAM	1 GB or higher (2 GB recommended)
Storage	5 GB for application, database, and logs
Network	Public IP / domain with HTTPS support for cloud deployment

2.2 Software Requirements

Software / Technology	Purpose
Windows 10/11 (or Linux / macOS)	Operating system for development
Android Studio	IDE for developing, building, and debugging the Android application
Java (JDK 17)	Programming language for the Android application

Software / Technology	Purpose
XML	Designing layouts and user interfaces
Node.js (LTS) and npm	Runtime and package manager for the backend server
Express.js	Web framework for building the REST APIs
MySQL 8.x and MySQL Workbench	Relational database and its design/administration tool
Firebase Cloud Messaging (FCM)	Sending push notifications to donors
Google Maps SDK / Location Services	Displaying maps and finding the user's location
Postman	Testing and documenting REST APIs
Git and GitHub	Version control and team collaboration
Notion	Project documentation and task tracking
Visual Studio Code	Code editor for backend development

2.3 Development Tools and Frameworks

Development was carried out using a set of widely adopted tools. **Android Studio** provides the Android emulator, layout editor, Gradle build system, and debugging tools required to build the client application. **Visual Studio Code** is used for the Node.js backend because of its lightweight editor, integrated terminal, and extensions for JavaScript and SQL.

On the backend, **Express.js** provides routing and middleware support. The main Node.js packages used are mysql2 (database connectivity with connection pooling), bcrypt (password hashing), jsonwebtoken (token-based authentication), express-validator (input validation), cors and helmet (cross-origin and header security), dotenv (environment configuration), and firebase-admin (sending FCM notifications).

On the Android side, **Retrofit and OkHttp** are used for communication with the REST API, **Gson** for converting JSON to Java objects, **RecyclerView** for lists, the **Google Maps SDK for Android** for maps, and the **Fused Location Provider** from Google Play Services for obtaining the device location. **Postman** is used to test each API endpoint independently of the mobile application, and **MySQL Workbench** is used to design the schema and run queries. **GitHub** hosts the source code, and **Notion** is used to maintain documentation, task lists, and review notes.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front end of the system is a native Android application written in **Java**, with user interfaces defined in **XML** layout files inside **Android Studio**. A native approach was chosen because the application depends heavily on device features such as GPS, background push notifications, and the Google Maps SDK, all of which are best supported on the native platform.

The interface follows Material Design guidelines. Screens are organised into Activities and Fragments: a splash and login screen, a registration screen, a donor dashboard, a recipient dashboard, a map screen, a request-creation form, a request-detail screen, a donation-history screen, and an admin panel. Lists of donors, requests, and history entries are displayed using RecyclerView with custom adapters. Forms validate input on the device (for example, mandatory blood group and a valid phone number) before sending data to the server, which gives immediate feedback and reduces unnecessary network calls.

Network communication is handled by Retrofit and OkHttp. The JSON Web Token received at login is stored on the device and attached to each request through an interceptor. The Google Maps SDK renders the map, and markers show the positions of donors, the requesting hospital, and nearby blood banks.

3.2 Back-End Design

The backend is developed with **Node.js** and **Express.js**. Node.js uses an event-driven, non-blocking model, which suits an application that handles many short requests such as logins, searches, and status updates. Express provides a clean routing layer and a middleware pipeline through which every request passes.

The backend code is organised into routes, controllers, services, and middleware. Routes map URLs to controller functions; controllers validate input and prepare responses; services contain the business logic such as blood-group compatibility, eligibility checks, and notification sending; and middleware performs authentication, role checking, validation, and error handling. This separation keeps the code readable and testable, and allows the user management, donor/request management, and notification modules to be separated into independent services if the system needs to scale.

The server exposes REST APIs that exchange JSON. Environment-specific values such as database credentials, the JWT secret, and the FCM service account are stored in environment variables and are never committed to the repository.

3.3 Database Design

MySQL is used as the primary database because the data in this application is naturally relational: a user may be a donor or a recipient, a recipient creates many blood requests, a

request receives many donor responses, and a donor has many donation records. MySQL provides primary and foreign keys, unique constraints, ENUM types, transactions, and indexing, all of which are required to maintain data integrity.

The schema is designed up to the **Third Normal Form (3NF)** to avoid redundancy and update anomalies. Authentication data is kept in a single users table, while donor-specific and recipient-specific attributes are stored in separate tables linked by foreign keys. A composite index on blood group, availability, and verification status speeds up donor searches. Distance between a request and a donor is calculated in SQL using the Haversine formula over the latitude and longitude columns. The complete schema is given in Chapter 5.

3.4 Version Control

Git and **GitHub** are used for version control and collaboration. The repository contains separate folders for the Android application and the backend server. The team follows a simple branching strategy: the main branch holds stable code, and each member works on a feature branch (for example, feature/donor-search or feature/fcm-notifications) which is merged through pull requests after review. Meaningful commit messages and regular commits make it possible to track changes, resolve conflicts, and revert faulty changes. Documentation and task tracking are maintained in **Notion**.

3.5 APIs or External Services

The application relies on the following external services:

- **Google Maps SDK for Android:** displays an interactive map with markers for donors, hospitals, and blood banks. An API key is created in the Google Cloud console and restricted to the application.
- **Google Location Services (Fused Location Provider):** obtains the current location of the user so that donors can be registered with accurate coordinates and recipients can search around their position.
- **Firebase Cloud Messaging (FCM):** delivers push notifications to donors when a compatible blood request is created. The backend sends messages using the Firebase Admin SDK, and the Android app receives them through a `FirebaseMessagingService`.
- **Postman:** used during development to test the APIs and share collections among team members.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram

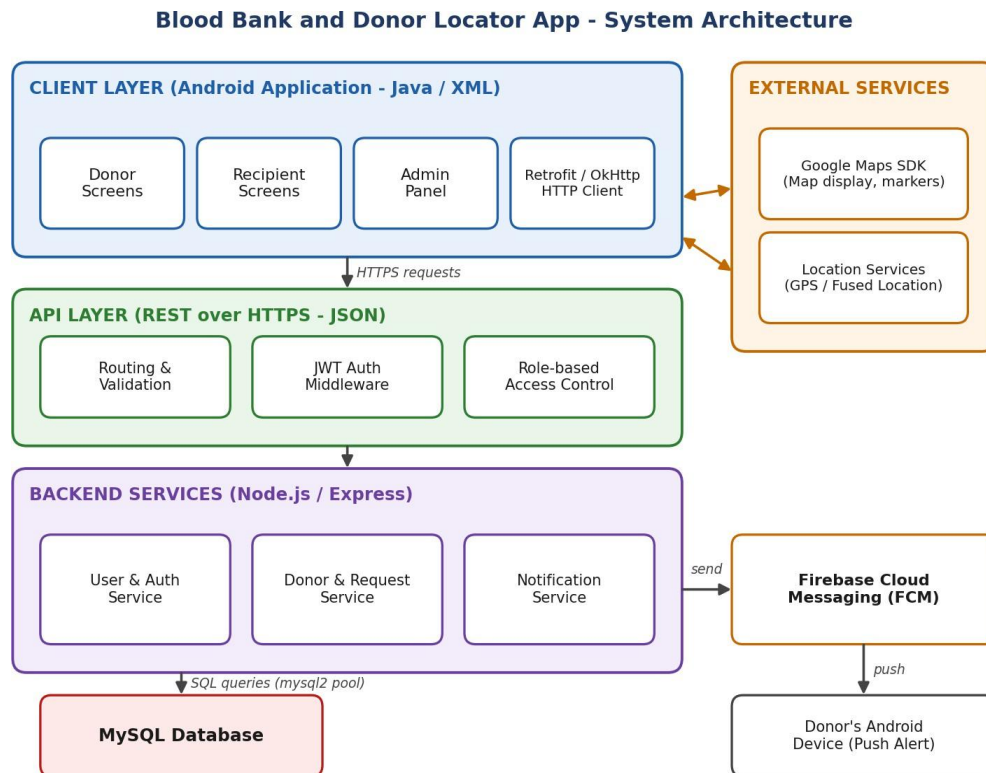


Figure 4.1: High-level architecture of the Blood Bank and Donor Locator App

4.2 Description of Each Layer (Frontend, Backend, Database)

The system follows a layered client–server architecture that separates the user interface, business logic, and data storage. Each layer communicates with the next only through well-defined interfaces, which makes the system easier to maintain and extend.

Client (Frontend) layer. The Android application provides the screens used by donors, recipients, and administrators. It collects input, validates it, requests the device location, displays maps, and calls the backend through Retrofit. It also hosts the `FirebaseMessagingService` that receives push notifications and opens the relevant request screen when the user taps an alert.

API layer. All client requests reach the backend through REST endpoints over HTTPS. Before a request reaches the business logic, it passes through middleware that validates the input, verifies the JSON Web Token, and checks that the user's role (donor, recipient, or admin) is allowed to use that endpoint.

Backend services layer. The Node.js/Express application contains three logical modules. The **User and Authentication Service** handles registration, login, and profile data. The **Donor and Request Service** manages donor profiles, blood requests, donor search, responses, and donation history. The **Notification Service** identifies matching donors for a new request and

sends alerts through FCM, recording each notification in the database. Because the modules communicate through clear functions and share no hidden state, they can be deployed as separate microservices if the load grows.

Database layer. MySQL stores all persistent data: users, donors, recipients, blood banks, blood requests, request responses, donation history, and notifications. The backend accesses it through a connection pool, and every query that includes user input uses parameterised statements to prevent SQL injection.

External services. The Google Maps SDK and location services are used directly by the Android client, while FCM is called from the backend, which then delivers the message to the donor's device.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

During development and demonstration, the architecture is deployed locally: the MySQL server and the Node.js application run on a team member's machine, and the Android application (running on an emulator or a physical device on the same network) connects to the local API address. This setup is simple and allows quick testing of every module.

For wider use, the same architecture can be deployed on the cloud. The Node.js backend can be hosted on a platform-as-a-service provider, the MySQL database on a managed MySQL instance, and the Android application distributed as a signed APK or through the Google Play Store. The backend communicates with FCM over the internet using a service account, and all client–server traffic is protected with HTTPS.

Source code is stored on GitHub. A continuous-integration workflow (for example, GitHub Actions) can be configured to install dependencies, run lint checks and tests on every pull request, and deploy the backend automatically when changes are merged into the main branch.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database schema

The database is designed around eight tables. The entity–relationship diagram below shows the tables, their primary keys (PK), foreign keys (FK), and the cardinality of each relationship.

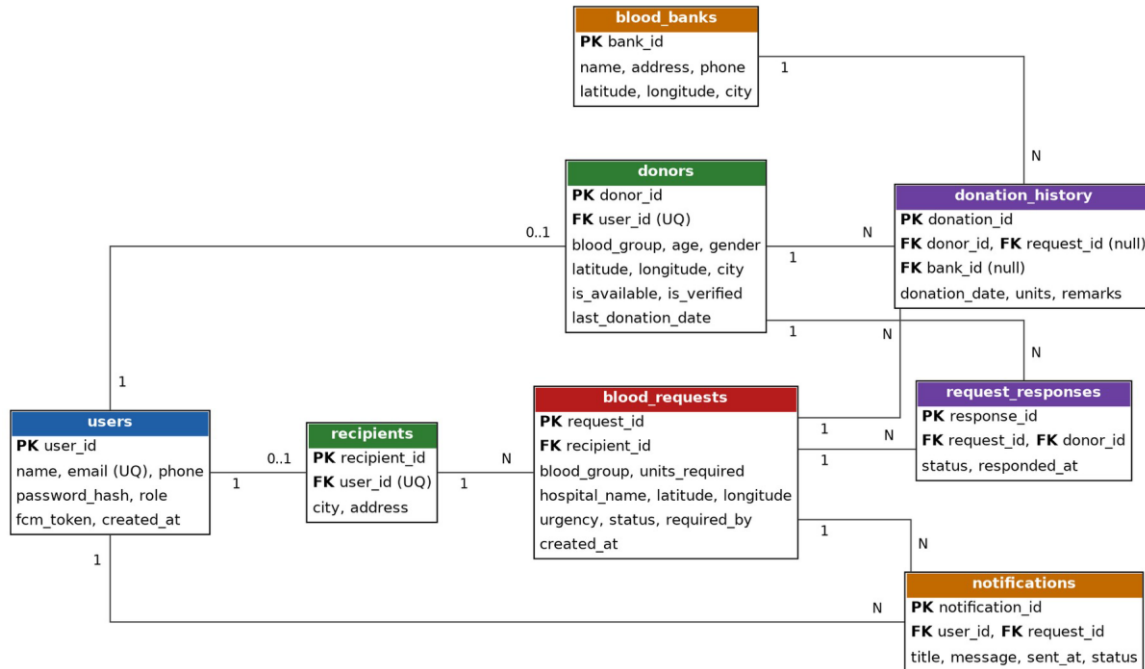


Figure 5.1: ER diagram of the Blood Bank and Donor Locator database

The relationships can be summarised as follows. One user may have at most one donor profile and at most one recipient profile (one-to-one). A recipient can create many blood requests (one-to-many). A blood request can receive many donor responses, and a donor can respond to many requests; the request_responses table resolves this many-to-many relationship. A donor can have many donation records, each optionally linked to the request it fulfilled and the blood bank where the donation took place. A user can receive many notifications, each linked to a request.

Database schema

Table	Columns	Keys and constraints
users	user_id, name, email, phone, password_hash, role, fcm_token, created_at	PK user_id; email UNIQUE; role ENUM(donor, recipient, admin)
donors	donor_id, user_id, blood_group, age, gender, latitude, longitude, city, is_available, is_verified, last_donation_date	PK donor_id; FK user_id (UNIQUE, ON DELETE CASCADE); index on (blood_group, is_available, is_verified)
recipients	recipient_id, user_id, city, address	PK recipient_id; FK user_id (UNIQUE, ON DELETE CASCADE)
blood_banks	bank_id, name, address, phone, latitude, longitude, city	PK bank_id

Table	Columns	Keys and constraints
blood_requests	request_id, recipient_id, blood_group, units_required, hospital_name, latitude, longitude, urgency, status, required_by, created_at	PK request_id; FK recipient_id; CHECK units_required 1 to 10; urgency and status ENUM
request_responses	response_id, request_id, donor_id, status, responded_at	PK response_id; FK request_id, donor_id; UNIQUE (request_id, donor_id)
donation_history	donation_id, donor_id, request_id, bank_id, donation_date, units, remarks	PK donation_id; FK donor_id; FK request_id and bank_id (nullable)
notifications	notification_id, user_id, request_id, title, message, sent_at, status	PK notification_id; FK user_id, request_id

All tables satisfy Third Normal Form: every non-key attribute depends on the key, the whole key, and nothing but the key. Donor-specific and recipient-specific attributes are separated from the authentication data in the users table, which avoids null-heavy rows and duplicated login information.

CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The application was implemented as a set of interrelated modules, each with a clear responsibility and a well-defined interface to the others. The work was divided among the team according to individual strengths, as shown below.

Team Member	Roll Number	Role
Simran Jangid	2520030166	Frontend / Android Developer
K Anusri	2520030432	Maps, Notifications and Admin Module
Venati Rukesh	2520030519	Backend and Database Developer

Module 1 – User Management and Authentication. This module handles registration and login. During registration, the user provides name, email, phone number, password, and role. The password is hashed with bcrypt before being stored, and the email must be unique. On login, the server verifies the credentials and returns a signed JSON Web Token that carries the user identifier and role. The Android app stores the token and sends it with every subsequent request.

Module 2 – Donor Management. A donor completes a profile consisting of blood group, age, gender, city, and current location (captured from the device GPS). The donor can switch availability on or off and the date of the last donation is recorded. New donor profiles are marked unverified until an administrator approves them; only verified and available donors appear in search results and receive notifications.

Module 3 – Recipient and Blood Request Management. A recipient creates a blood request by entering the required blood group, number of units, hospital name and location, urgency level, and the date by which blood is needed. The request is stored with the status open and can later be marked accepted, fulfilled, or cancelled. Recipients can view and manage all their requests.

Module 4 – Donor Search and Maps. The search service first determines which donor blood groups are compatible with the requested group, then queries the database for donors who are available, verified, and eligible (a minimum of 90 days since the last donation) within a chosen radius. Distance is computed in SQL using the Haversine formula, and results are sorted from nearest to farthest. The Android client plots the results on a Google Map and also in a list showing distance and a call option.

Module 5 – Notification Service. When a request is created, the notification service selects matching donors, builds an FCM message containing the blood group, hospital, and required date, sends it through the Firebase Admin SDK, and stores a record in the notifications table. If a device token is invalid or expired, the failure is logged and the token is cleared.

Module 6 – Donation History. When a request is fulfilled, or when a donor records a donation at a blood bank, an entry is added to `donation_history`, and the donor's `last_donation_date` is updated. Donors can view their complete history, and the date is used to enforce the eligibility interval in future searches.

Module 7 – Admin Panel. Administrators can view all users, donors, and requests; verify or reject donor profiles; deactivate or delete accounts that are fake or misused; and monitor request statuses. All admin endpoints are protected by role-based middleware.

6.2 Front-end logic (UI rendering, state management)

Each screen in the Android application is implemented as an Activity or Fragment with an XML layout. When a screen opens, it reads any required data (for example, the stored token or the selected request identifier), calls the relevant API through Retrofit, and renders the response. List screens such as the donor-search results, request list, and donation history use RecyclerView with adapters that bind each JSON object to a row layout.

Application state is kept simple and explicit. The authentication token, user role, and basic profile information are stored in SharedPreferences (using encrypted preferences for the token) so that the user remains logged in between sessions. Screen-level state, such as the selected blood group and search radius, is held in the Activity or in a ViewModel so that it survives screen rotation. Network calls run asynchronously and update the interface through callbacks, with progress indicators displayed while the request is in progress and clear error messages shown when it fails.

Location and maps are handled through the Fused Location Provider and the Google Maps SDK. The app requests location permission at runtime; if the user denies it, the user can enter the city or pick a point on the map manually. Markers are added for each donor in the search result, and tapping a marker shows the donor's blood group, distance, and a call button. A custom FirebaseMessagingService receives push notifications and, when the user taps one, opens the request-detail screen directly.

6.3 API Integration

The Android application communicates with the backend exclusively through REST APIs that exchange JSON over HTTP(S). The main endpoints are summarised below. All endpoints other than registration and login require a valid JWT in the Authorization header.

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register a new donor, recipient, or admin	Public
POST	/api/auth/login	Authenticate and receive a JWT	Public
PUT	/api/users/me/fcm-token	Save or refresh the device's FCM token	Logged-in user
POST	/api/donors/profile	Create or update the donor profile	Donor

Method	Endpoint	Purpose	Access
PUT	/api/donors/me/availability	Switch availability on or off	Donor
GET	/api/donors/search	Search compatible donors by blood group, location, radius	Recipient / Admin
POST	/api/requests	Create a blood request and notify donors	Recipient
GET	/api/requests	List requests (own requests or open requests nearby)	Logged-in user
PUT	/api/requests/:id/status	Update status (accepted, fulfilled, cancelled)	Recipient / Admin
POST	/api/requests/:id/respond	Donor accepts or declines a request	Donor
GET	/api/donations/history	View donation history	Donor
POST	/api/donations	Record a donation	Donor / Admin
GET	/api/bloodbanks	List nearby blood banks	Logged-in user
GET	/api/admin/users	List all users	Admin
PUT	/api/admin/donors/:id/verify	Verify or reject a donor	Admin
DELETE	/api/admin/users/:id	Remove a user account	Admin

Responses follow a consistent format: a suitable HTTP status code (200, 201, 400, 401, 403, 404, 500) and a JSON body containing either the requested data or a message describing the error. This consistency allows the Android client to handle errors in a single place.

6.4 Authentication and Authorisation

Security is implemented at several levels. **Password protection:** passwords are never stored in plain text; they are hashed with bcrypt using a salt before being saved. **Authentication:** after a successful login, the server issues a JSON Web Token signed with a secret key and having a limited validity period. The token is verified by an authentication middleware on every protected route, and requests with a missing, tampered, or expired token receive a 401 response.

Authorisation: a role-checking middleware compares the role stored in the token with the roles allowed for the endpoint. For example, only a recipient can create a blood request, only a donor can update availability, and only an administrator can verify donors or delete users; any other attempt receives a 403 response.

Input validation and injection protection: all request bodies and query parameters are validated (for example, blood group must be one of the eight valid values and coordinates must be numeric). Every SQL statement uses parameterised queries, so user input is never concatenated into SQL. **Transport and header security:** the production deployment uses HTTPS, and the helmet and cors packages restrict headers and origins. **Privacy:** a donor's phone number is exposed only to authenticated users who are searching, and donors can disable availability at any time.

CHAPTER 7- FEATURES

7.1 List of Main Features

S. No	Feature	Description
1	Secure registration and login	Role-based accounts for donors, recipients, and admins using JWT authentication
2	Donor profile and availability	Blood group, location, last donation date, and an availability toggle
3	Blood request creation	Recipients post requests with blood group, units, hospital, urgency, and date
4	Compatible donor search	Search by blood-group compatibility, availability, eligibility, and distance
5	Map-based view	Donors, hospitals, and blood banks displayed on Google Maps
6	Instant notifications	FCM push alerts to matching donors when a request is posted
7	Donor response	Donors accept or decline requests; recipients see who responded
8	Donation history	A record of every donation for each donor
9	Nearby blood banks	List and map of blood banks near the user
10	Admin panel	Verification of donors and management of users and requests

7.2 How Each Feature Works

1. *Secure registration and login*

User perspective: the user chooses a role, enters basic details and a password, and is able to log in immediately afterwards. The app remembers the session until the user logs out. **Technical description:** the register endpoint validates the data, hashes the password with bcrypt, and inserts a row in the users table. The login endpoint compares the password with the stored hash and, if it matches, returns a signed JWT containing the user identifier and role.

2. *Donor profile and availability*

User perspective: the donor fills in blood group, age, gender, and city, taps a button to capture the current location, and later uses a simple switch to show whether they are available to donate. **Technical description:** the profile is saved in the donors table with latitude and longitude from the Fused Location Provider. The availability toggle updates the is_available column, and only donors with is_available and is_verified set to true are returned by searches.

3. *Blood request creation*

User perspective: a recipient opens a form, selects the blood group, number of units, urgency, and required date, and picks the hospital location on the map. After submission, the request appears in the recipient's list with the status open. **Technical description:** the POST

/api/requests endpoint validates the payload, inserts a row in blood_requests, and then triggers the notification service asynchronously so that the response is not delayed.

4. Compatible donor search

User perspective: the recipient selects a blood group and a radius and instantly sees a list and map of suitable donors sorted by distance. **Technical description:** the service maps the requested blood group to the list of compatible donor groups (for example, an A negative recipient can receive from A negative and O negative). It then runs a single SQL query that filters by those groups, availability, verification, and a 90-day gap since the last donation, and calculates the distance with the Haversine formula.

5. Map-based view

User perspective: donors and blood banks appear as markers; tapping a marker shows the blood group, distance, and a call button. **Technical description:** the Google Maps SDK renders the map; the Android client adds a marker for each coordinate returned by the API and adjusts the camera to include all markers.

6. Instant notifications

User perspective: a matching donor receives a notification on the phone, even when the app is closed, and can tap it to open the request. **Technical description:** after a request is created, the backend gets the FCM tokens of matching donors and sends a multicast message using the Firebase Admin SDK. A row is stored for each notification, and expired tokens are removed when FCM reports them as invalid.

7. Donor response

User perspective: the donor taps Accept or Decline on the request screen; the recipient sees the list of donors who accepted and can call them. **Technical description:** the respond endpoint inserts or updates a row in request_responses (unique per request and donor) and changes the request status to accepted when the first donor accepts.

8. Donation history

User perspective: the donor sees a timeline of past donations with date, place, and units. **Technical description:** when a request is marked fulfilled or a donation is recorded, a row is added to donation_history and the donor's last_donation_date is updated within a database transaction so that both changes succeed or fail together.

9. Nearby blood banks

User perspective: a user can view blood banks around them with their address and phone number. **Technical description:** the blood_banks table stores coordinates, and the same distance calculation used for donor search returns the nearest banks.

10. Admin panel

User perspective: an administrator sees pending donors, approves or rejects them, and can block fake or misused accounts. **Technical description:** admin endpoints are protected by the role middleware; verification updates `is_verified`, and deletion removes the user, with related rows removed through `ON DELETE CASCADE`.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Each API endpoint was tested independently in Postman before being connected to the Android application. A Postman collection was created with folders for authentication, donors, requests, donations, blood banks, and admin operations. An environment variable stores the base URL and the JWT, so that after logging in the token is automatically included in the Authorization header of later requests. For every endpoint, both a success scenario and failure scenarios (missing fields, invalid values, missing token, and wrong role) were tried.

Request	Input	Expected Status
POST /api/auth/register	Valid user details	201 Created
POST /api/auth/register	Email already registered	409 Conflict
POST /api/auth/login	Correct email and password	200 OK with token
POST /api/auth/login	Wrong password	401 Unauthorized
GET /api/donors/search	Valid blood group, lat, lng, radius	200 OK with donor list
GET /api/donors/search	Invalid blood group	400 Bad Request
POST /api/requests	Valid request, recipient token	201 Created
POST /api/requests	Donor token (wrong role)	403 Forbidden
GET /api/admin/users	No token	401 Unauthorized

Front-end testing was performed by running the Android application on an emulator and on a physical device and checking that each screen displays the data returned by the API, that validation messages are shown for invalid input, and that the application handles network errors gracefully.

8.2 Integration Testing

Integration testing verified that the modules work together as complete flows rather than in isolation. The following end-to-end scenarios were defined:

1. **Registration to login:** a donor registers in the app, logs in, and the token is stored and used to open the dashboard.
2. **Donor onboarding:** the donor completes the profile, an admin verifies it, and the donor then appears in search results.
3. **Request to notification:** a recipient creates a request; matching donors receive an FCM notification within a few seconds, and a notification record is saved.
4. **Response to fulfilment:** a donor accepts the request, the recipient sees the response, the request is marked fulfilled, and the donation appears in the donor's history with an updated last-donation date.

5. **Eligibility rule:** after the donation above, the same donor no longer appears in searches until the minimum gap has passed.

8.3 Tools Used for Testing

Tool	Purpose
Postman	Testing REST APIs, saving collections, and automating checks with test scripts
MySQL Workbench	Inspecting tables and verifying data after each operation
Android Emulator	Running the app on virtual devices with mock locations
Physical Android device	Testing GPS, push notifications, and real network conditions
Firebase Console	Sending test notifications and checking delivery
Browser developer tools / server logs	Inspecting responses and debugging errors

8.4 Test Cases and Results

The test cases below cover the main functions of the system. The Result column should be updated with Pass or Fail after each case is executed on the final build.

ID	Test Scenario	Expected Result	Result
TC01	Register with valid details	Account created; message shown; user can log in	To be recorded
TC02	Register with an existing email	Error: email already registered	To be recorded
TC03	Login with valid credentials	Token issued; dashboard opens	To be recorded
TC04	Login with wrong password	Error message; no token issued	To be recorded
TC05	Open protected API without token	401 Unauthorized	To be recorded
TC06	Donor turns availability off	Donor no longer appears in search	To be recorded
TC07	Recipient creates a valid request	Request saved with status open	To be recorded
TC08	Create request with invalid blood group	400 error with validation message	To be recorded
TC09	Search donors for B-negative within 10 km	Only B-negative and O-negative donors in range, nearest first	To be recorded
TC10	Search when donor is unverified	Unverified donor is excluded	To be recorded
TC11	Search for donor who donated 30 days ago	Donor is excluded (90-day rule)	To be recorded
TC12	Notification on new request	Matching donors receive a push alert	To be recorded
TC13	Donor accepts a request	Response saved; recipient sees acceptance	To be recorded
TC14	Mark request fulfilled	Donation history entry added; last donation date updated	To be recorded
TC15	Admin verifies a donor	is_verified becomes true	To be recorded
TC16	Donor calls an admin endpoint	403 Forbidden	To be recorded

ID	Test Scenario	Expected Result	Result
TC17	Admin deletes a user	User and linked rows removed	To be recorded
TC18	Location permission denied	App offers manual location selection	To be recorded
TC19	Map screen with search results	Markers displayed for each donor	To be recorded
TC20	SQL injection text in a search field	Input rejected or treated as plain text; no data exposed	To be recorded

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

Database

1. Install MySQL 8.x and create a database named bloodbank_db.
2. Run the schema script (tables, keys, and indexes) from MySQL Workbench or the command line.
3. Insert sample data for blood banks and, if required, a default administrator account.

Backend server

1. Install Node.js (LTS) and clone the backend folder from the GitHub repository.
2. Run npm install to download the dependencies.
3. Create the .env file with the database credentials, JWT secret, and port number.
4. Place the Firebase service-account key file in the secure configuration location.
5. Start the server with npm start (or a process manager such as PM2 for production) and verify the health endpoint.

Android application

1. Open the Android project in Android Studio and let Gradle sync.
2. Add the Google Maps API key and the google-services.json file generated from the Firebase console.
3. Set the BASE_URL constant to the address of the running backend.
4. Build a debug APK for testing, or a signed release APK for distribution, and install it on the device.

9.2 Environment Configuration

Setting	Where	Description
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	Backend .env	MySQL connection details
PORT	Backend .env	Port on which Express listens
JWT_SECRET, JWT_EXPIRES_IN	Backend .env	Secret key and validity of authentication tokens
FIREBASE_SERVICE_ACCOUNT	Backend config	Service-account credentials for sending FCM messages
MAPS_API_KEY	Android manifest / local.properties	Key for the Google Maps SDK, restricted to the app
google-services.json	Android app folder	Firebase project configuration for FCM
BASE_URL	Android build config	Address of the backend API

Secrets such as the database password, the JWT secret, and the service-account key are never committed to GitHub; the .env file and key files are listed in .gitignore. A separate configuration is used for development and production so that test data does not mix with real data.

9.3 Hosting of Frontend and Backend

Frontend: the Android application is distributed as an APK file. For academic demonstration, the APK is installed directly on the evaluators' or team's devices. For public release, it can be signed with a release key and published on the Google Play Store.

Backend and database: for demonstration, the Node.js server and MySQL run on a local machine reachable from the test devices through the local network. For wider access, the backend can be hosted on a cloud application platform and the database on a managed MySQL service, with a domain name and an HTTPS certificate. Environment variables are configured in the hosting dashboard, and the server restarts automatically on failure. The Android application is then configured with the public API address.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

1. **Accurate distance calculation:** calculating the distance between a request and many donors efficiently required a suitable formula and indexing strategy.
2. **Blood-group compatibility:** the compatibility rules are not symmetric (for example, O negative can donate to all groups but can receive only from O negative), so a simple equality check on blood group was not sufficient.
3. **Push-notification delivery:** device tokens change when the app is reinstalled or data is cleared, and notifications failed for such stale tokens.
4. **Location on emulators and permission handling:** emulators do not provide real GPS data by default, and users may deny location permission.
5. **Connecting the app to the local server:** the emulator could not reach the server through localhost, and plain HTTP traffic was blocked by default on newer Android versions.
6. **Data reliability:** donor data can become outdated or may be entered incorrectly or falsely.
7. **Team coordination:** several members worked on the app, backend, and database at the same time, which led to merge conflicts and mismatches in API formats.

10.2 Solutions Applied

1. The Haversine formula was used directly in a SQL query, and a composite index on blood group, availability, and verification reduced the number of rows to be evaluated.
2. A lookup table of compatible donor groups was implemented in the service layer and covered by test cases for all eight blood groups.
3. The app uploads the new token whenever onNewToken is called, and the server removes tokens that FCM reports as invalid.
4. Mock locations were used for emulator testing, and a manual location option was provided when permission is denied.
5. The emulator was pointed at the host machine's special address, and a network security configuration was used for development, while production uses HTTPS.
6. A donor verification step by the admin, an availability toggle, and a minimum gap between donations were added to keep data reliable.
7. Feature branches, pull requests, a shared Postman collection, and an agreed API specification were used to reduce conflicts.

10.3 Current Limitations or Known Bugs

- The app depends on the donor keeping their location and availability updated; it does not track the donor continuously.
- Donor verification is done manually by the administrator and does not include medical screening or document validation.
- Blood-bank stock is maintained manually and is not synchronised in real time with hospital systems.
- Push notifications require an internet connection and Google Play Services; there is no SMS fallback.
- The application is currently available only for Android devices and in the English language.
- Distance is calculated in a straight line (as the crow flies) and not by actual road distance or travel time.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

- **AI-based donor matching:** ranking donors by distance, past response rate, and availability patterns, and predicting who is most likely to respond.
- **Hospital and blood-bank integration:** connecting with hospital and blood-bank systems so that real-time stock is visible and requests can be raised directly by hospitals.
- **Expansion to more cities and regions:** supporting multiple cities, regional languages, and state-level administration.
- **Integration with additional healthcare services:** ambulance services, organ and plasma donation, and health-camp information.
- **Donation-camp management:** allowing organisations to announce camps and notify nearby donors.
- **Rewards and reminders:** badges for regular donors and reminders when they become eligible again.

11.2 Possible Integrations or Optimisations

- **Microservices:** splitting the user, donor/request, and notification modules into independent services behind an API gateway.
- **Caching and spatial indexing:** using a cache for frequent searches and spatial indexes to speed up location queries as the user base grows.
- **Message queues:** sending notifications through a queue so that large broadcasts do not slow down request creation.
- **SMS and WhatsApp fallback:** reaching donors who have no internet connection.
- **Government blood-bank portals:** integration with national blood-bank directories for wider stock visibility.
- **CI/CD and monitoring:** automated tests, deployments, and monitoring dashboards for production reliability.
- **iOS and web versions:** extending the platform to other devices using the same backend APIs.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Blood Bank and Donor Locator App addresses a real and urgent problem: the difficulty of finding compatible blood donors quickly. The project designs and develops an Android application supported by a Node.js and Express backend and a MySQL database. Donors register their blood group, location, and availability; recipients create blood requests and search for compatible donors nearby using Google Maps; and matching donors are alerted instantly through Firebase Cloud Messaging.

The system maintains a complete donation history and provides an administrator panel for verifying donors and managing requests. Security is ensured through hashed passwords, JWT authentication, role-based authorisation, input validation, and parameterised queries. The architecture is layered and modular, so that it can be extended with additional features or split into microservices in the future.

12.2 What Was Achieved

- A centralised platform that connects donors and recipients.
- A normalised MySQL database with tables for users, donors, recipients, blood banks, requests, responses, donation history, and notifications.
- REST APIs for authentication, donor management, request management, search, donation history, and administration.
- Location-based donor search using blood-group compatibility, availability, eligibility, and distance.
- Instant push notifications to suitable donors.
- An admin panel for controlling users and requests.
- Version-controlled source code and structured documentation.

12.3 Skills Learned During Development

- Relational database design, normalisation, constraints, indexing, and writing SQL queries including geospatial calculations.
- Building REST APIs with Node.js and Express, including middleware, validation, and error handling.
- Implementing authentication and authorisation with bcrypt and JWT.
- Android development with Java and XML, Retrofit, RecyclerView, location services, and the Google Maps SDK.
- Integrating Firebase Cloud Messaging for real-time notifications.
- Testing APIs with Postman and designing test cases.

- Teamwork, version control with Git and GitHub, and documentation with Notion.

CHAPTER -13 REFERENCES

Books

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Tutorials and Learning Resources

MDN Web Docs. (2026). HTTP and JavaScript Documentation. Retrieved from <https://developer.mozilla.org>

W3Schools. (2026). SQL Tutorial. Retrieved from <https://www.w3schools.com>

GeeksforGeeks. (2026). Node.js, Express.js, and Android Development Tutorials. Retrieved from <https://www.geeksforgeeks.org>

Tutorialspoint. (2026). Android and MySQL Tutorials. Retrieved from <https://www.tutorialspoint.com>

APIs and Tools Used

Google Maps Platform. (2026). Maps SDK for Android Documentation. Retrieved from <https://developers.google.com/maps>

Firebase by Google. (2026). Firebase Cloud Messaging Documentation. Retrieved from <https://firebase.google.com/docs/cloud-messaging>

Android Developers. (2026). Android Studio and Platform Documentation. Retrieved from <https://developer.android.com>

Postman. (2026). Postman Learning Center. Retrieved from <https://learning.postman.com>

GitHub. (2026). GitHub Docs. Retrieved from <https://docs.github.com>

Documentation and Research References

Node.js Foundation. (2026). Node.js Documentation. Retrieved from <https://nodejs.org/docs>

Express.js. (2026). Express Routing and Middleware Guide. Retrieved from <https://expressjs.com>

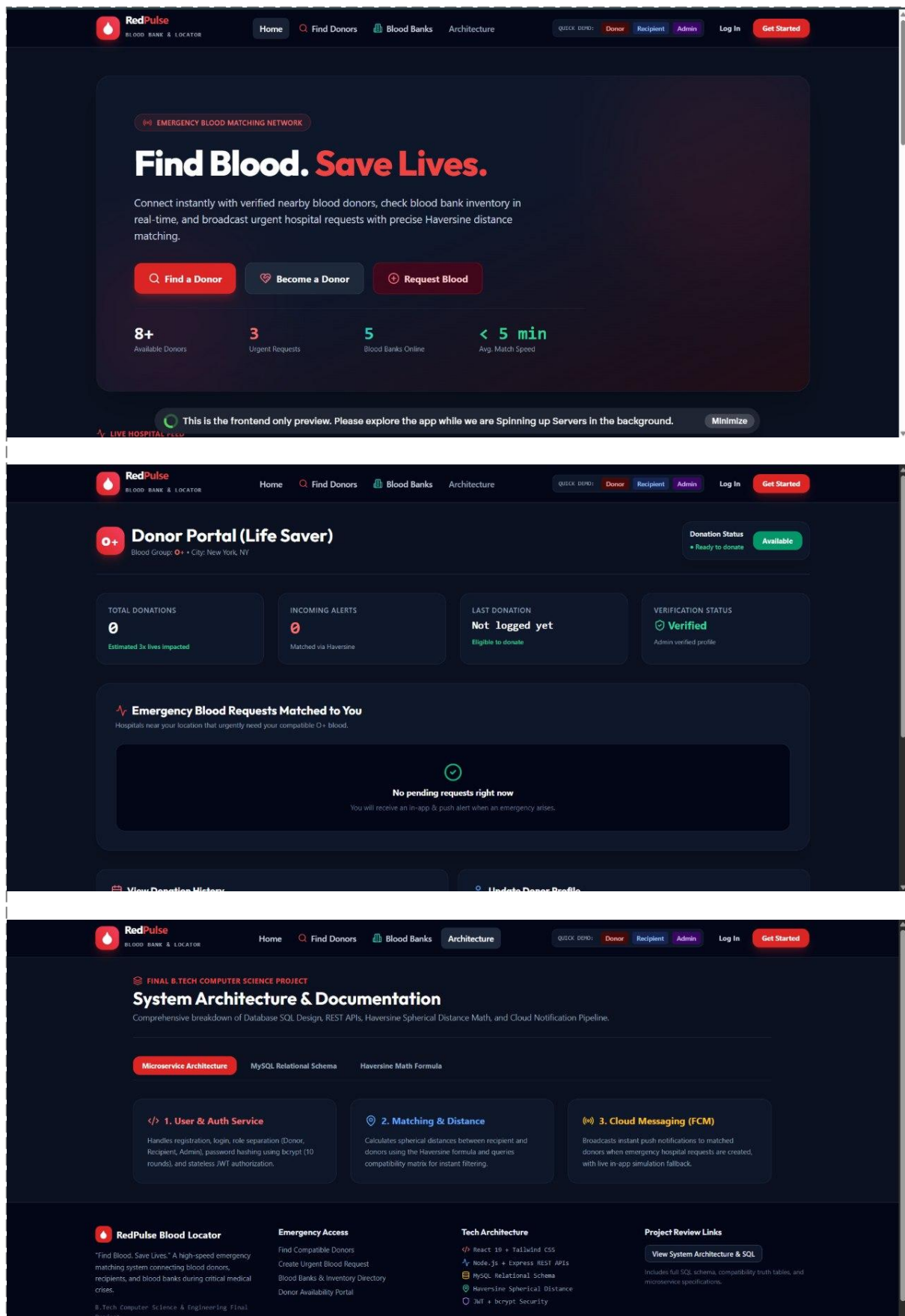
Oracle Corporation. (2026). MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force. Retrieved from <https://www.rfc-editor.org/rfc/rfc7519>

World Health Organization. (2026). Blood Safety and Availability. Retrieved from <https://www.who.int>

CHAPTER -14 APPENDICES

Screenshots of the app



Sample code snippets

1. Database schema (excerpt)

```
CREATE TABLE users (  
  user_id      INT AUTO_INCREMENT PRIMARY KEY,  
  name         VARCHAR(100) NOT NULL,  
  email        VARCHAR(120) NOT NULL UNIQUE,  
  phone        VARCHAR(15)  NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role         ENUM('donor','recipient','admin') NOT NULL,  
  fcm_token     VARCHAR(255) NULL,  
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE donors (  
  donor_id      INT AUTO_INCREMENT PRIMARY KEY,  
  user_id       INT NOT NULL UNIQUE,  
  blood_group    ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NULL,  
  age           TINYINT UNSIGNED NOT NULL,  
  gender        ENUM('male', 'female', 'other') NOT NULL,  
  latitude       DECIMAL(9,6) NOT NULL,  
  longitude      DECIMAL(9,6) NOT NULL,  
  city          VARCHAR(60),  
  is_available   BOOLEAN DEFAULT TRUE,  
  is_verified    BOOLEAN DEFAULT FALSE,  
  last_donation_date DATE NULL,  
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,  
  INDEX idx_blood_available (blood_group, is_available, is_verified)  
);  
  
CREATE TABLE blood_requests (  
  request_id     INT AUTO_INCREMENT PRIMARY KEY,  
  recipient_id   INT NOT NULL,  
  blood_group     ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NULL,  
  units_required TINYINT UNSIGNED NOT NULL CHECK (units_required BETWEEN 1 AND 10),  
  hospital_name  VARCHAR(120) NOT NULL,  
  latitude       DECIMAL(9,6) NOT NULL,  
  longitude      DECIMAL(9,6) NOT NULL,  
  urgency        ENUM('normal', 'urgent', 'critical') DEFAULT 'normal',  
  status         ENUM('open', 'accepted', 'fulfilled', 'cancelled') DEFAULT 'open',  
  required_by    DATE NOT NULL,  
  created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (recipient_id) REFERENCES recipients(recipient_id) ON DELETE CASCADE  
);
```

2. Compatible donor search API (Node.js / Express)

```
// routes/donors.js - compatible donor search (Haversine distance in SQL)  
const express = require('express');  
const pool = require('../config/db');  
const { authenticate } = require('../middleware/auth');  
const router = express.Router();  
  
const COMPATIBLE_DONORS = {  
  'A+': ['A+', 'A-', 'O+', 'O-'],  
  'A-': ['A-', 'O-'],  
  'B+': ['B+', 'B-', 'O+', 'O-'],
```

```

    'B-': ['B-', 'O-'],
    'AB+': ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'],
    'AB-': ['A-', 'B-', 'AB-', 'O-'],
    'O+': ['O+', 'O-'],
    'O-': ['O-']
  };

  router.get('/search', authenticate, async (req, res) => {
    const { bloodGroup, lat, lng, radius = 10 } = req.query;
    const groups = COMPATIBLE_DONORS[bloodGroup];
    if (!groups || isNaN(lat) || isNaN(lng)) {
      return res.status(400).json({ message: 'Invalid blood group or location' });
    }
    const sql =
      'SELECT d.donor_id, u.name, u.phone, d.blood_group, d.latitude, d.longitude, ' +
      '(6371 * ACOS(LEAST(1, COS(RADIANS(?)) * COS(RADIANS(d.latitude)) * ' +
      'COS(RADIANS(d.longitude) - RADIANS(?)) + SIN(RADIANS(?)) * SIN(RADIANS(d.latitude))))' +
      ' AS distance_km ' +
      'FROM donors d JOIN users u ON u.user_id = d.user_id ' +
      'WHERE d.is_available = 1 AND d.is_verified = 1 AND d.blood_group IN (?) ' +
      'AND (d.last_donation_date IS NULL OR d.last_donation_date <= DATE_SUB(CURDATE(), INTERVAL' +
      '90 DAY)) ' +
      'HAVING distance_km <= ? ORDER BY distance_km ASC LIMIT 50';
    try {
      const [rows] = await pool.query(sql, [lat, lng, lat, groups, Number(radius)]);
      res.json({ count: rows.length, donors: rows });
    } catch (err) {
      console.error(err);
      res.status(500).json({ message: 'Server error' });
    }
  });
};

module.exports = router;

```

3. Notification service using Firebase Cloud Messaging

```

// services/notificationService.js - alert matching donors through FCM
const admin = require('firebase-admin');
const pool = require('../config/db');

admin.initializeApp({ credential:
admin.credential.cert(require('../serviceAccountKey.json')) });

async function notifyDonors(request, donors) {
  const tokens = donors.map(d => d.fcm_token).filter(Boolean);
  if (tokens.length === 0) return { sent: 0 };

  const message = {
    tokens: tokens,
    notification: {
      title: 'Urgent blood request: ' + request.blood_group,
      body: request.hospital_name + ' needs ' + request.units_required + ' unit(s) by ' +
request.required_by
    },
    data: { requestId: String(request.request_id), type: 'BLOOD_REQUEST' }
  };

  const result = await admin.messaging().sendEachForMulticast(message);

  for (const d of donors) {

```

```

        await pool.query(
            'INSERT INTO notifications (user_id, request_id, title, message, status) VALUES
            (?, ?, ?, ?, ?)',
            [d.user_id, request.request_id, message.notification.title, message.notification.body,
            'sent']
        );
    }
    return { sent: result.successCount, failed: result.failureCount };
}

module.exports = { notifyDonors };

```

4. Android: Retrofit interface and Firebase messaging service

```

// ApiService.java - Retrofit interface used by the Android app
public interface ApiService {
    @POST("api/auth/login")
    Call<LoginResponse> login(@Body LoginRequest body);

    @GET("api/donors/search")
    Call<DonorSearchResponse> searchDonors(
        @Header("Authorization") String bearerToken,
        @Query("bloodGroup") String bloodGroup,
        @Query("lat") double lat,
        @Query("lng") double lng,
        @Query("radius") int radiusKm);

    @POST("api/requests")
    Call<RequestResponse> createRequest(
        @Header("Authorization") String bearerToken,
        @Body BloodRequest body);
}

// MyFirebaseService.java - receives push alerts and refreshes the FCM token
public class MyFirebaseService extends FirebaseMessagingService {
    @Override
    public void onNewToken(String token) {
        // send the new token to PUT /api/users/me/fcm-token
        TokenUploader.upload(getApplicationContext(), token);
    }

    @Override
    public void onMessageReceived(RemoteMessage message) {
        String title = message.getNotification().getTitle();
        String body = message.getNotification().getBody();
        String requestId = message.getData().get("requestId");
        NotificationHelper.show(this, title, body, requestId); // opens RequestDetailActivity
    }
}

```

Installation/setup instructions

The Blood Bank and Donor Locator App has two parts that must be set up: the backend server with its database, and the Android application.

Backend and database

1. Install Node.js (LTS), MySQL 8.x, and Git on the computer.
2. Clone the project repository from GitHub and open the backend folder in a terminal.
3. Create a database named `bloodbank_db` and run the provided schema script.
4. Copy the sample environment file to `.env` and enter the database credentials, JWT secret, and port.
5. Run `npm install`, then `npm start`. The terminal should show that the server is listening and connected to the database.

Android application

1. Install Android Studio and open the Android project folder.
2. Add the Google Maps API key and the `google-services.json` file from the Firebase console.
3. Set the `BASE_URL` to the backend address (for an emulator, the host machine's special address; for a physical device, the computer's local IP address).
4. Click Run to install the app on an emulator or on a connected phone with USB debugging enabled.

User manual or guide

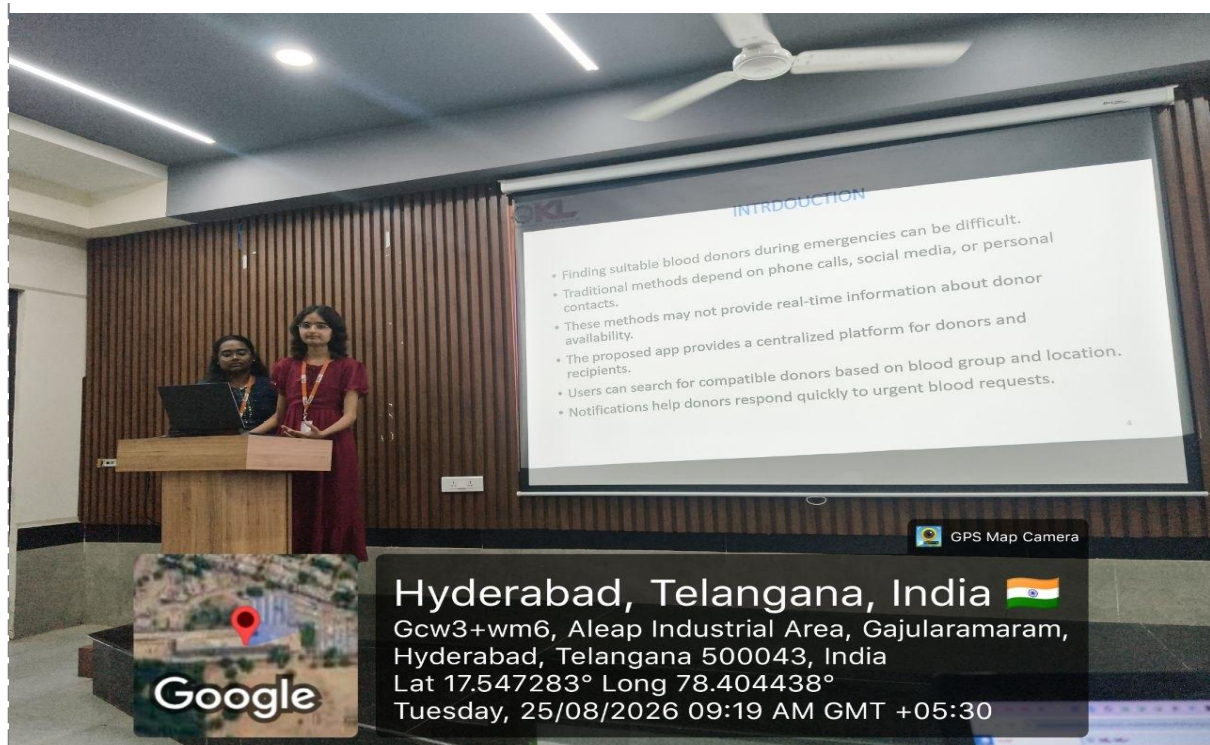
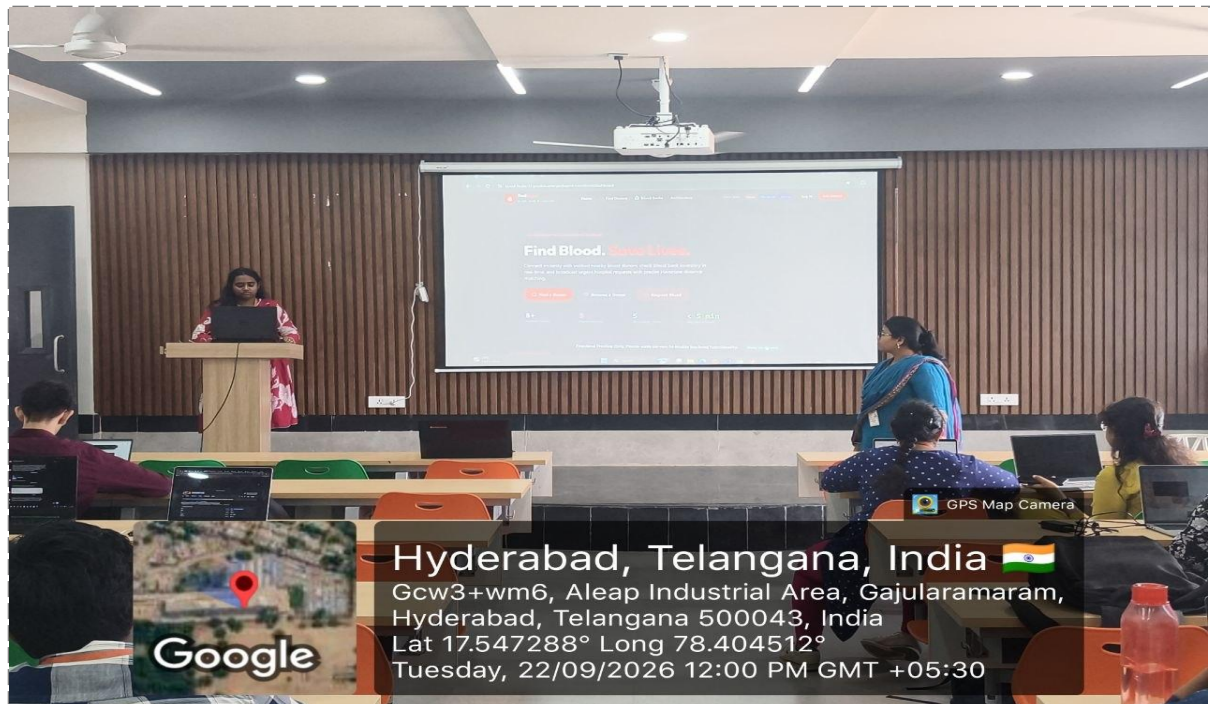
Registering: open the app, tap Register, choose whether you are a donor or a recipient, fill in your details, and create a password. Log in with your email and password.

Donors: complete your profile by selecting your blood group, age, gender, and city and tapping Use Current Location. Wait for the administrator to verify your profile. Use the availability switch on the dashboard to show whether you can donate. When a nearby request matches your blood group, you will receive a notification; tap it to see the details and choose Accept or Decline. Your past donations are shown under Donation History.

Recipients: tap Create Request, select the blood group, number of units, urgency, hospital location, and date, and submit. Open Search Donors to view compatible donors in a list or on the map, and use the call button to contact them. When the blood has been arranged, mark the request as fulfilled or cancelled.

Administrators: log in with the admin account to view users and requests, verify new donors, and remove fake or misused accounts.

Geo Tag photos with guide



Review forms with guide signatures

Insert scanned review forms with the guide's signatures here.